Team144_ESA_Report_Phase2.docx

2,741 Words

16% Matches

1	Internet	
www.ijirem.org		4%
2	Internet	
www.coursehero.com		4%
3	Internet	
www.scribd.com		3%
4	Internet	
isteonline.in		1%
5	Internet	
es.slideshare.net		<1%
6	Internet	
avscollege.ac.in		<1%
7	Internet	
krify.co		<1%
8	Internet	
www.studymode.com		<1%
9	Internet	
www.bennetts.co.uk		<1%
10	Internet	
department-of-computer-science-engineering.newhorizoncollegeofengineering.in		<1%

Dissertation on

"AVIOS – Adapative Vigiliant Intellient OS"

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

UE22CS320B – Capstone Project Phase - 2

Submitted by:

KURUMETI PRAJVAL	**PES2UG22CS274**
MANOJ KUMAR KS	**PES2UG22CS302**
DHANUSHREE K	**PES2UG22CS176**
NAGARATNA	**PES2UG22CS331**

Under the guidance of

Prof. Swetha Patil
PES University

January - May 2025

Electroni City, Hosur 100,

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled

"AVIOS – Adapative Vigiliant Intellient OS"

is a bonafide work carried out by

KURUMETI PRAJVAL	**PES2UG22CS274**
MANOJ KUMAR KS	**PES2UG22CS302**
DHANUSHREE K	**PES2UG22CS176**
NAGARATNA	**PES2UG22CS331**

In partial fulfillment for the completion of sixth-semester Capstone Project Phase - 2 (UE22CS320B) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan. 2025 – May. 2025. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The dissertation has been approved as it satisfies the 6th-semester academic requirements in respect of project work.

Signature Prof. Swetha Patil Assistant Professor	Signature Dr. Sandesh B J Chairperson	Signature Dr. B K Keshavan Dean of Faculty

External Viva

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

DECLARATION

We hereby declare that the Capstone Project Phase - 2 entitled **"AVIOS – Adaptative Vigilant Intelligent Operating System"** has been carried out by us under the guidance of **Prof. Swetha Patil, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester January – May 2025. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

PES2UG22CS274	**KURUMETI PRAJVAL**
PES2UG22CS302	**MANOJ KUMAR KS**
PES2UG22CS176	**DHANUSHREE K**
PES2UG22CS331	**NAGARATNA**

ACKNOWLEDGEMENT

I would like to express my gratitude to **Prof. Swetha Patil,** Department of Computer Science and Engineering, PES University, for his/ her continuous guidance, assistance, and encouragement throughout the development of this UE22CS320B - Capstone Project Phase – 2

I am grateful to all Capstone Project Coordinators, for organizing, managing, and helping with the entire process.

I take this opportunity to thank Dr. Sandesh B J, Professor & Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department. I would like to thank Dr. B.K. Keshavan, Dean of Faculty, PES University for his help.

I am deeply grateful to Dr. M. R. Doreswamy, Chancellor, PES University, Prof. Jawahar Doreswamy, Pro-Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University, and Prof. Nagarjuna Sadineni, Pro-Vice Chancellor, PES University, for providing me with various opportunities and enlightenment every step of the way. Finally, Phase 1 of the project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

In our day-to-day life computers play an important role. As we are moving into AI era we need our systems to be the best at their performance. For this the systems should utilise the CPU and i/o resources to the best. For that efficient CPU and i/o scheduling is essential for their maximizing system performance. with the help of Al and ML model we can able to analyse the user behaviour and try to improve the system performance.

Operating system is the backbone of computational system we will see how integration of in OS makes the systems smarter and adoptable. Traditional operating systems manage hardware and software well but they don't have the intelligence to improve the user experience or make operations more efficient on their own with the help AIOS we can improve user experience by context aware search, personalized recommendations, intelligent allocations and predictive system maintenance. Using techniques such as neural networks pattern recognition and fuzzy logic, AI OS aims to manage system management and saves user time and also improve overall performance

In this project we are trying to improve the CPU scheduling of a Linux system by understanding the user behaviour. we are using two models. One is to classify the task type and another one is to predict the CPU time [process execution time]. For this we have considered static and dynamic properties of the new processes in the queue. With the help of the dataset, we have classified the task into two types they are resource intensive task and real time tasks. Along with that we also predict the CPU execution time of task and place it in respective scheduling algorithms.

With the help of predicted time slice and task type we will assign the nice value for it and make that task execute it faster so that it will help in utilising the resources at their best.AI model which are built for prediction are placed at user level and with the help of system calls we will communicate with the kernel to schedule appropriate algorithm to the task.

TABLE OF CONTENTS

(This page is left blank on purpose)

CHAPTER 1: INTRODUCTION

In this era, Operating systems have become the core of our daily competitive experience. Linux, Has One of the most widely used operating system as it is the open source continuous to evolve rapidly to meet the increasingly complex needs of users [1]. One of the main problem user faces is how to increase productivity and efficiency in using the applications within the Operating System [2][3]. Although Linux offers high flexibility and control, users offer encounter difficulties in finding and using the applications that suits for their needs.[4].

Artificial Intelligence operating system plays an important role in the transformation step in Operating system.

Where we integrate AI technologies into OS and enable intelligent automation productive analysis and increase system Performance. Unlike Traditional OS architecture, AI OS try to identify patterns, context- aware processing and personalization to deliver a highly responsive and adaptive computer environment. These are designed to Operate across different platform like Personal computers, IOT devices and enterprise network make them versatile and future ready.

AIOS aims to solve these challenges by developing an intelligent operating system capable of providing applications recommendations based on individual usage patterns and performance. The system will use artificial intelligence technology to analyze applications usage data and user behavior pattern. [5][6]. Thus, this system can help Linux users choose and use applications more effectively, accordingly to their daily needs.

In modern era, where multitasking, real time responsiveness, and optimal resource utilization are need of existing scheduling are not by for the best. and also, traditional schedulers use predefined static rules and fails to learn user patterns. By integrating AL into the core of the OS, we can try to shift towards a more intelligent and predictive model and help in improving system performance.

This study aims to integrate the power of AI with the flexibility of Linux by creating an adaptive, intelligent operating system. The goal is that to improve application scheduling and helps in optimize resource use, and system operations by identifying problems in the application and management this not only tackles immediate user needs but also pushes forward the integration of AI in open-source systems. The aim is to build smart and more responsive computing environment that aligns with the future of operating systems design and functionalities

CHAPTER 2: PROBLEM STATEMENT

Designing a Model for improving CPU Scheduling by using Machine Learning in Linux. Integrating AI into operating system brings excellent opportunities but also challenging traditional operating system are built to handle fixed tasks and static user needs. They lack the adaptability and intelligence to respond dynamically changing user behaviours, changing application demands and evolving hardware setups. This becomes more evident as computing environments to grow more complex and diverse, Ranging from personal devices and IoT platforms to large scale systems.

In today's computing world user expect operating system to do more than manage resources efficiently, they want systems that can anticipate their needs, boost productivity performance and simplify interactions. However, most OS architecture doesn't have built in mechanism intelligent decision making and automation. While some systems have introduced basic AI driven features like Voice recognition and task prediction, these implementations are often limited by scope and functionalities.

By integrating AI in OS, we can get benefits like:

- Resource Management: AI can help in dynamic allocation of system resources based on the workloads and increases efficiency and overall system performance.
- Adaptive system behaviour: AI integrated Operating system can evolve overtime by learning from user interpretations to better meet changing needs and application demands.

It also helps in reducing the context switch between process and help in increase the system performance.

CHAPTER 3: DATA

4.1 Overview

In this project, we are working with system metrics data collected from Linux based environment. The primary objective of data is to get real time data which helps us in making an efficient ready to use models that can integrated into OS easily. This data helps us in analyses of system behaviour on different conditions. To enhance the number of records we incorporated data generated using GAN model with the dataset we collected from real time data of Linux systems. Additionally, we are using the Linux based Virtual machine data and Google Cluster Brog traces 2019 dataset which are publicly available on internet sources. This diverse dataset helps us in creating a solid foundation for building up the AI model.

4.2 Dataset

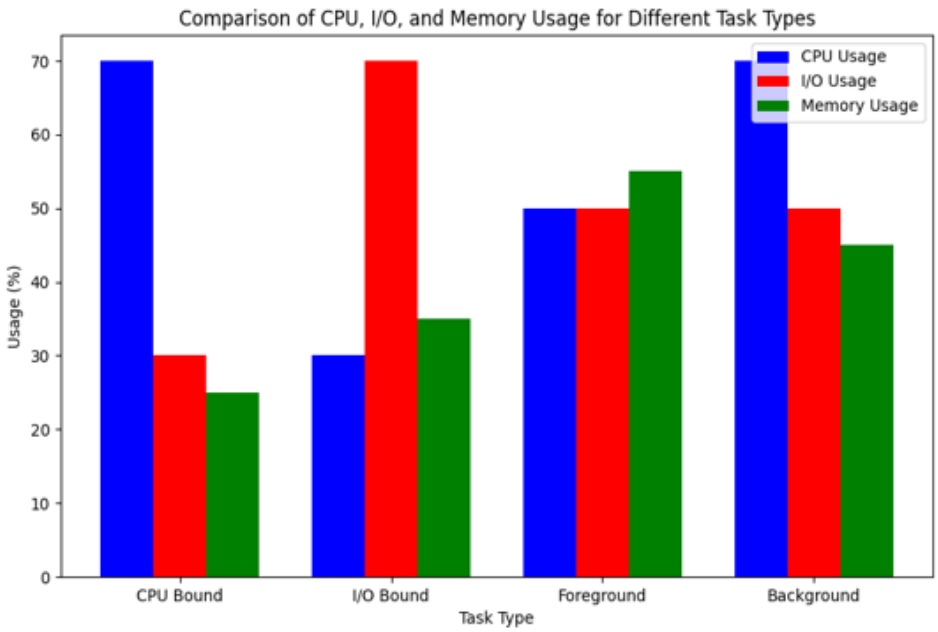
- **Realtime Linux system data**
  We collected real time system activity dat from the computer labs of PES University. This involved monitoring the system metric for a perod of time which resulted in 10000 to 12000 records of raw data. After reviewing and cleaning the data we have been left over with 4000 to 5000 records of dataset.
- **Synthetic Data Generation using GAN**
  Using the cleaned 4000 to 5000 data we created a GAN model that creates a synthetic Dataset which is approximately 200000 records of dataset.
- **Additional Dataset**
  - Linux Virtual machine Dataset
  - Google Cluster Brog Traces Dataset

4.3 Data Preprocessing

Before feeding the data to train a AI model preprocessing is a very crucial step. These are the following steps were taken to ensue we have a quality and consistency of dataset:

- **Normalization**
  All numerical features were brought to consistent range (0 to 1), facilitating better model performance and faster convergence during training process.
- **Feature Selection**
  After reviewing the dataset we have removed all the useless or irrelevant features to reduce noise in dataset and improve the model performance

Data visualization: Task vs Avg. Usage (CPU, memory, io)



CHAPTER 4: DESIGN DETAILS

4.1 Novelty:

AVIOS involves the integration of **AI with Kernel-level scheduling,** which is an unique approach. Unlike traditional OS where it uses a static scheduler, AVIOS leverages on AI to handle the dynamic, data-driven decision to predict the CPU time slices in real time.

4.2 Innovativeness:

The system includes dual space design one for user and the other Kernel. In user space it handles all the task to be classified by AI such as I/O type, CPU intensive, Memory intensive task and also predicts the Vruntime for a particular task. In kernel space it enforces scheduling through system calls.

4.3 Interoperability:

AVIOS is built in such a way that it allows the smooth operation with popular tools like perf, psutil, ftrace and it can be even extended to work with docker and Kubernetes as well. This interoperability supports deployment across different environments such as cloud, edge and desktop.

4.4 Performance:

AVIOS helps to improve the performance by reducing latency, boosting CPU utilization and prioritizing the critical task. The performance can be verified by using ml models like Random Forest, XG Boost where the accuracy achieved is about the 95.01% in task classification, efficient scheduling and resource allocation for the identified task.

4.5 Security:

As security is most concerned feature the scheduler implements strict access controls, which permits only privileged processes to modify the task priorities as per the requirements. Usage of safe system call modification protects the kernel from unauthorized interference from AI models.

4.6 Reliability:

AVIOS helps to boost the system reliability via real-time-monitoring, adaptive dynamic scheduling. The model feeds on the past data and updates itself to improve the fault tolerance over time.

4.7 Maintainability:

The system is maintained and designed well since it uses two space one for user and the other for kernel. AI components are operated entirely in the user space without disturbing the kernel. But the user and kernel space use the system calls () for further communication.

4.8 Portability:

AVIOS is built using python, C, TensorFlow, PyTorch which run smoothly across all the modern Linux multi-core processors. So we can easily migrate the system between different environments without any delay.

4.9 Legacy to Modernization:

AVIOS helps to upgrade the existing Linux scheduler which uses the CFS and SCHED_RR for task scheduling, Avios is helping to overcome the static scheduling method with rigid policies, by making use of the modern technologies like AI to dynamically schedule the task based on the priority by making smarter decisions which is more required for the future.

4.10 Reusability:

The system is built in such a way that, its architecture and AI models aren't just fixed to single use they be extended to other domains like large data centers, mobile operating systems, robotics, edge computing and etc. We can extend its implementation to the fields where ever a dynamic scheduling is required, since it can be reused across different systems without any hurdle.

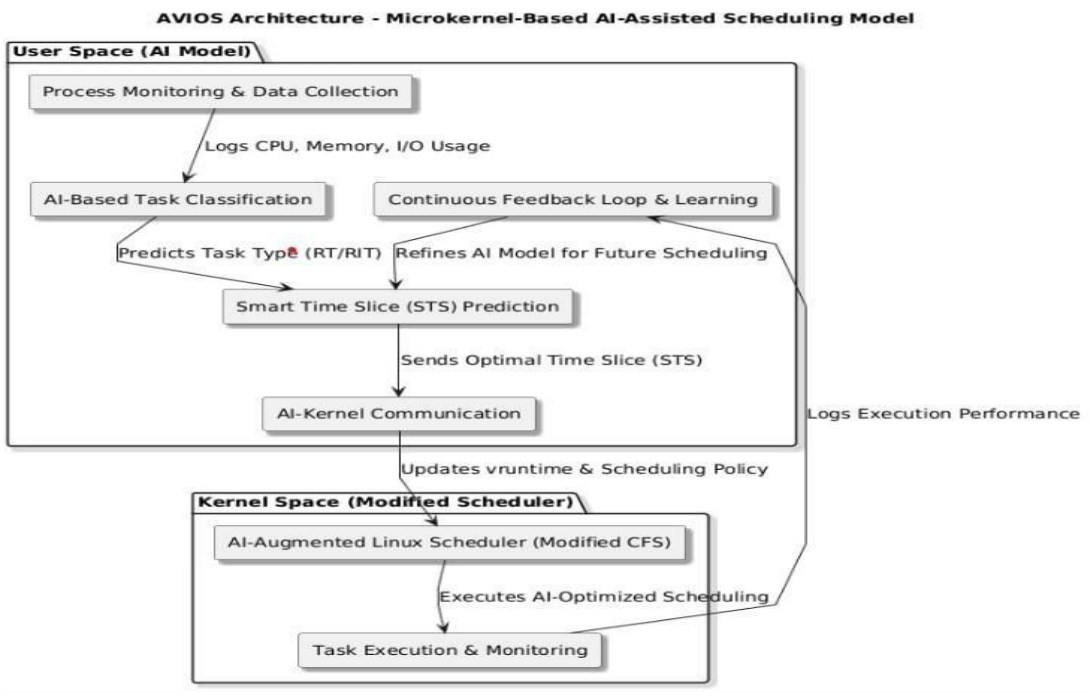
4.11 Application Compatibility:

A major strength of AVIOS is, It easily fits into existing systems and it easily blends with the current applications, without modifying the user programs. Since it uses the application layer, the transition is seamless, and everything stays fully compatible with backward.

4.12 Resource Utilization:

AVIOS constantly monitors CPU, memory and I/O activity to utilizes the system resources smartly. This is achieved by reducing the background noise and avoiding unnecessary task migrations, This not only helps the smooth functioning of the system but also helps to save energy.

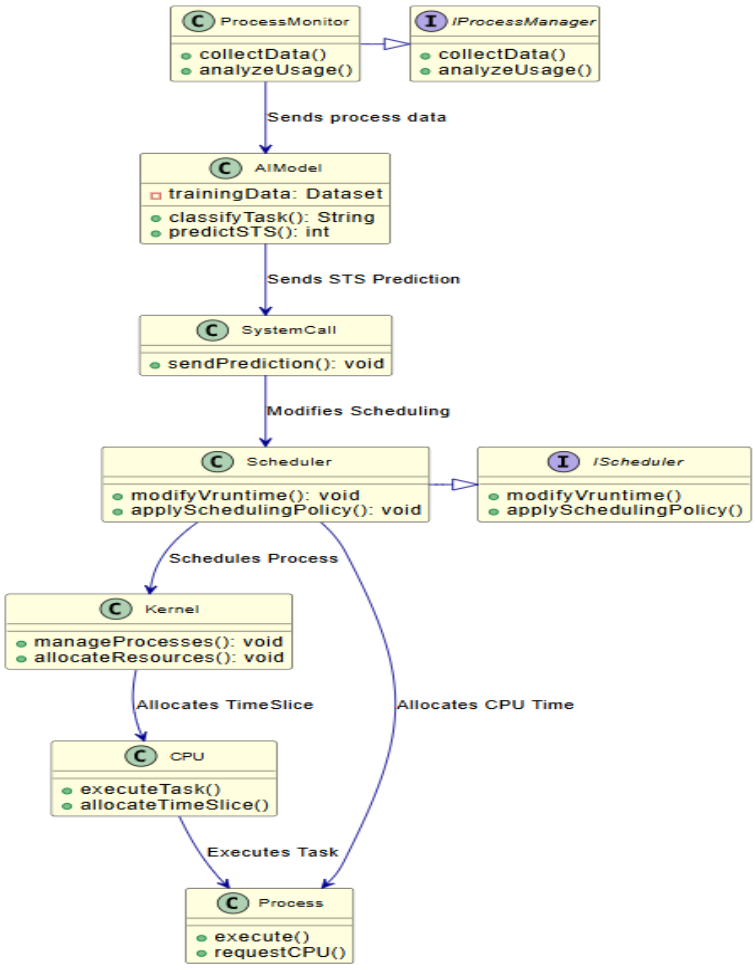
CHAPTER 5: HIGH LEVEL SYSTEM DESIGN/ SYSTEM ARCHITECTURE



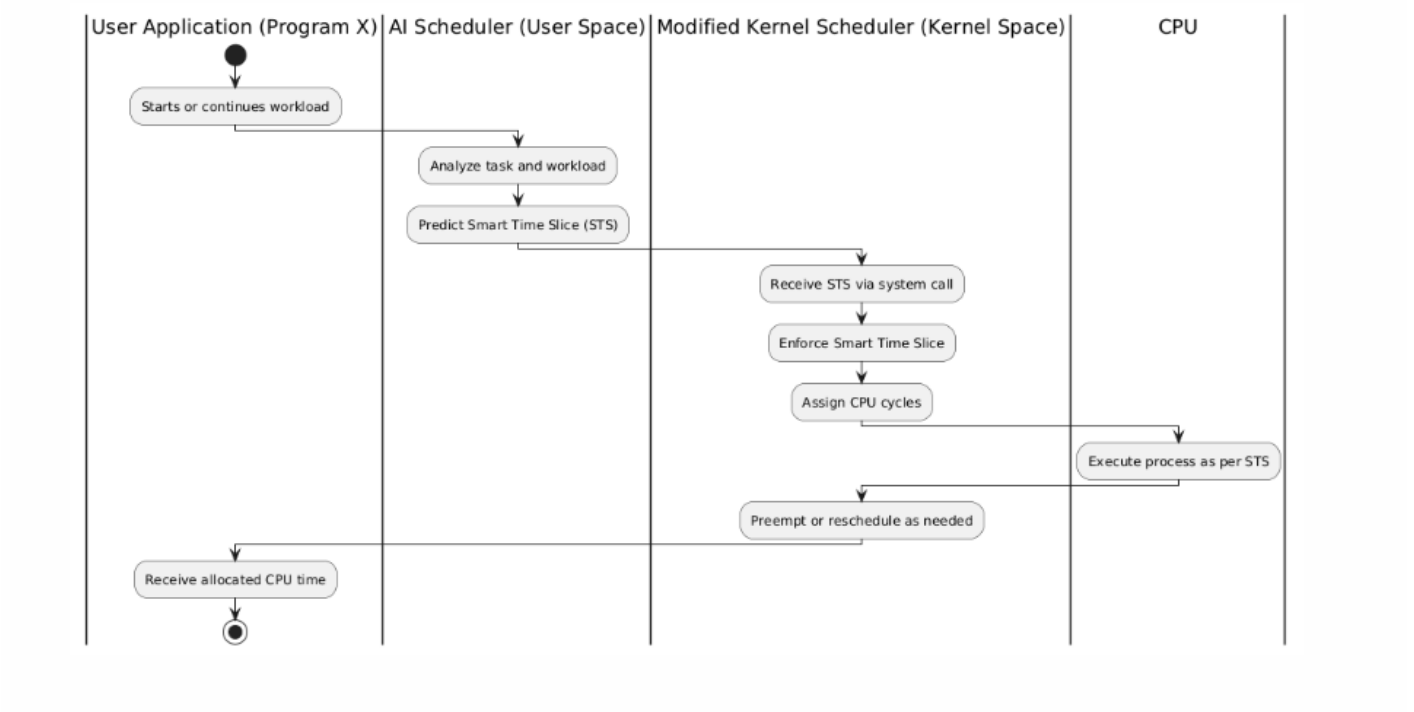
This architecture follows a **Microkernel-based AI-Assisted Scheduling** approach. The **AI Model** in user space that classifies tasks and analyzes the workload of **Program X** and predicts an optimal **Smart Time Slice (STS)**. It then makes a **system call** to the **Modified Scheduler** in kernel space, which dynamically adjusts CPU allocation based on AI recommendations. The scheduler enforces the AI-determined time slice and assigns CPU cycles accordingly. This approach improves resource efficiency by integrating **machine learning-based adaptive scheduling** while keeping the kernel minimal and modular.

CHAPTER 6: DESIGN DESCRIPTION

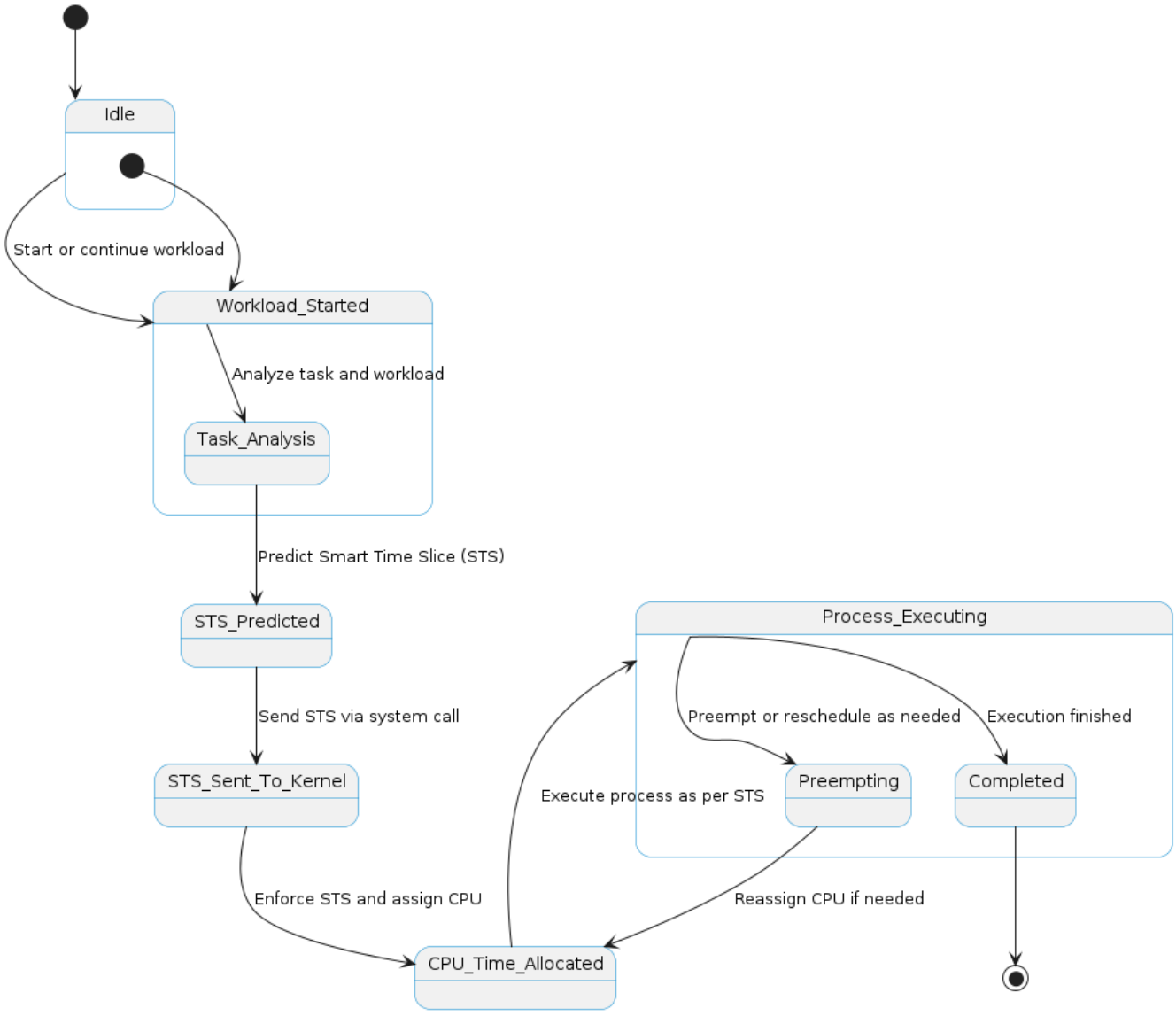
6.1 Master Class Diagram



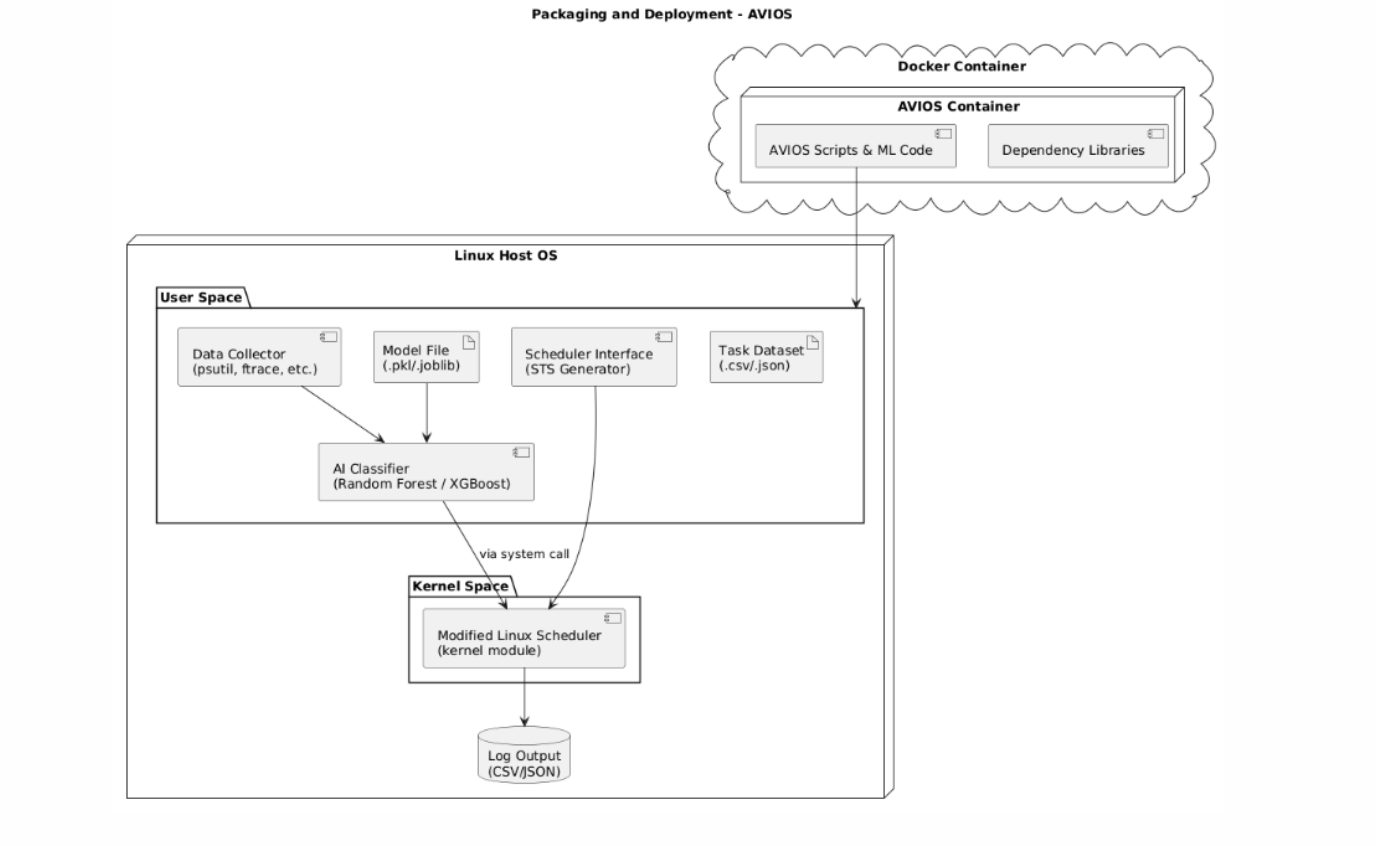
6.2 Swimlane Diagram



6.3 State Diagram



6.4 Packaging and Deployment Diagram



CHAPTER 7: TECHNOLOGIES USED

- **Python**: A versatile, high-level programming language popular for scripting, web development, data science, and AI.
- **C:** A powerful, low-level programming language used for system programming, operating systems, and embedded systems.
- **Scikit-Learn (sklearn):** A Python library offering simple and efficient tools for data mining, machine learning, and predictive analysis.
- **Linux Kernel:** The core part of the Linux operating system, managing hardware, processes, memory, and system calls.
- **TensorFlow:** An open-source machine learning framework developed by Google for building and deploying deep learning models.
- **PyTorch:** An open-source deep learning library developed by Meta, known for its dynamic computation graph and flexibility in AI research.

CHAPTER 8: IMPLEMENTATION AND PSEUDOCODE

Module Overview

The goal of this module is to build and train a machine learning model that can classify system tasks based on how they behave — like how much CPU, memory, or I/O they use. This classification helps our AVIOS scheduler decide how to allocate CPU time more efficiently.
We first load the task dataset and clean it by removing any unnecessary columns and missing data. We then prepare the data by encoding text labels into numbers and scaling all feature values to a standard range.
Since our original dataset had some imbalance (some task types were underrepresented), we used a technique called SMOTE to create a more balanced version of the data. This makes the model perform better across all types of tasks.
Finally, we trained some models — Random Forest, XGBoost , Support Vector Machine (SVM) — to predict the task type. We used both accuracy and cross-validation scores to check how well the models are performing.
The trained model (especially Random Forest, which gave us better accuracy) is used later in the system to predict the Smart Time Slice (STS) that should be given to each process.

Pseudocode

Below is the simplified step-by-step explanation of what the program does:

1. Read the task data from a CSV file
2. Remove any rows with missing values
3. Drop columns that are not useful for model training (e.g., Task ID, Name, etc.)
4. Separate the input features (X) from the target label (Y)
5. Convert task types (text) into numerical labels using label encoding
6. Normalize all numerical features using standard scaling
7. Balance the dataset using SMOTE to make sure all task types are equally represented
8. Split the balanced data into training and test sets (90% training, 10% testing)
9. Train a Random Forest Classifier using the training data
   → Make predictions using the test data
   → Calculate model accuracy
   → Perform 5-fold cross-validation and print average accuracy

Return trained model so they can be used in the main AVIOS system

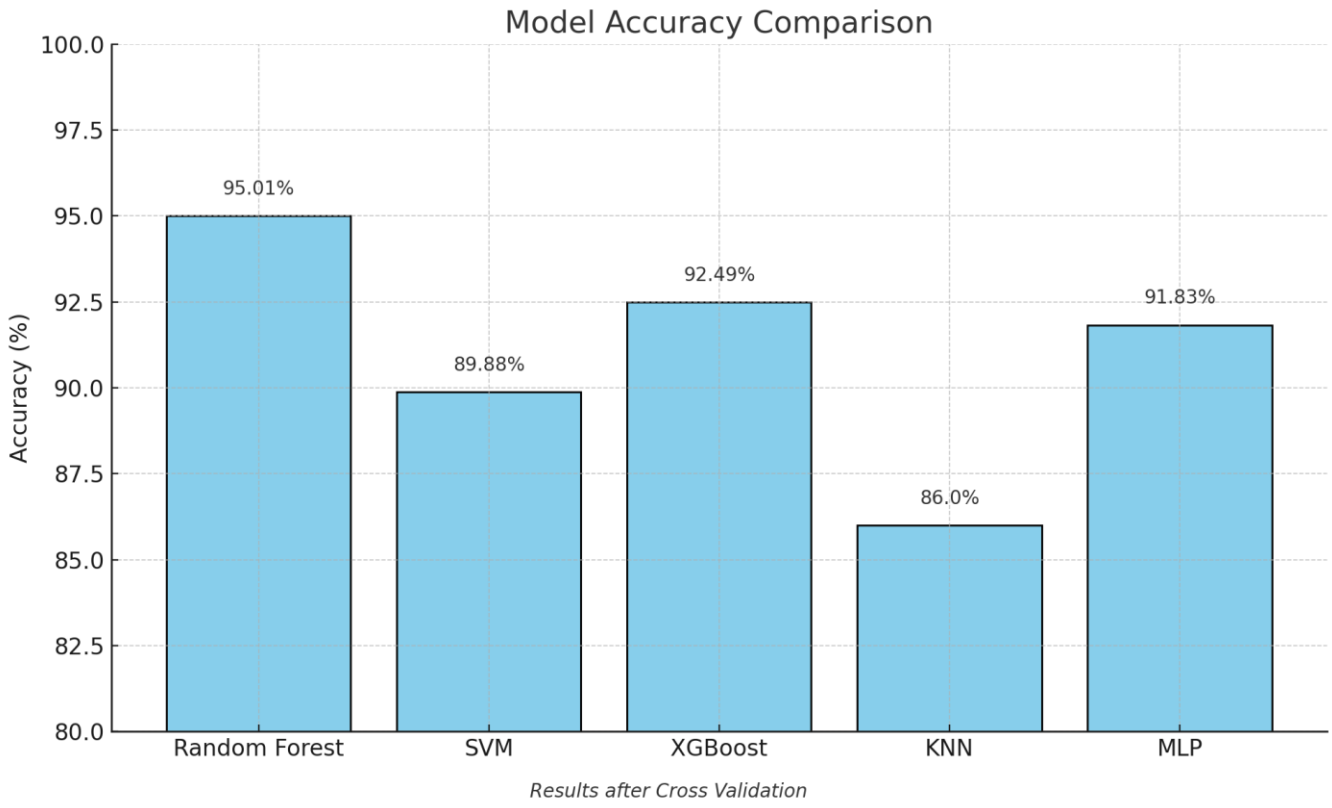
CHAPTER 9: CONCLUSION OF CAPSTONE PROJECT PHASE – 2

In this phase of the Capstone project, we focused on building the core logic of our system — AVIOS, an AI-based intelligent task scheduler. The main aim was to create a system that could analyze the behavior of tasks in real-time and assign CPU time based on how demanding the task is. Instead of relying on static scheduling logic, our system learns from system-level data and uses that to make smarter decisions.

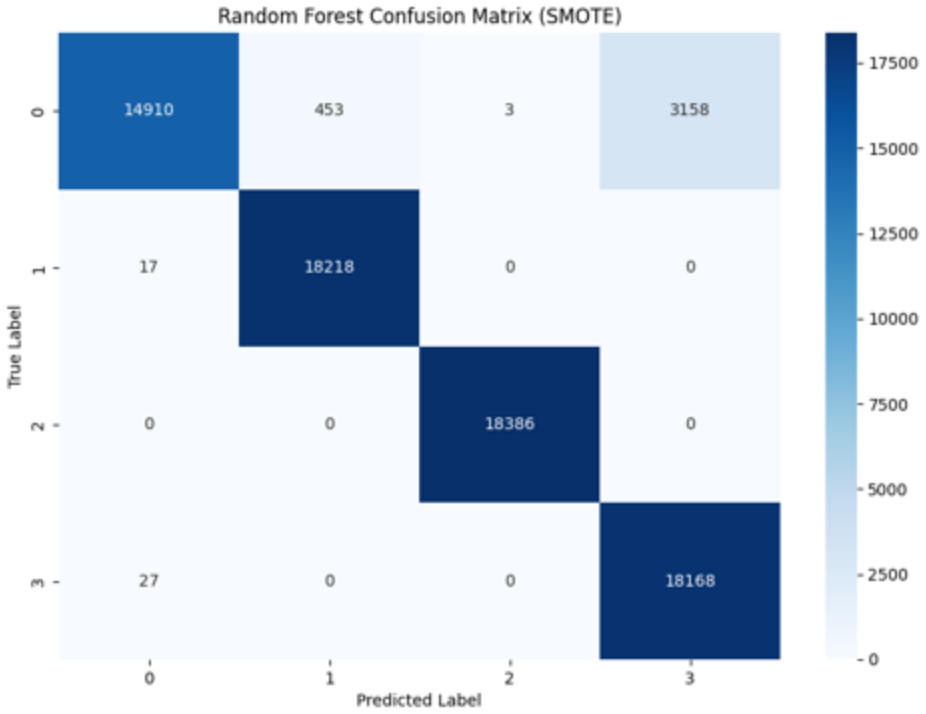
We started by collecting performance metrics like CPU usage, memory, I/O load, execution time, and task priority using various tools like psutil, perf, and ftrace. After cleaning and preparing the data, we trained machine learning models — Random Forest and SVM — to classify tasks as CPU-bound, I/O-bound, or background/foreground. These predictions help the kernel scheduler adjust the time slice given to each task, which leads to better performance and system responsiveness.
To address the class imbalance in our dataset, we used SMOTE to generate balanced samples, and also experimented with GANs to synthesize more data when needed. We trained our models using this improved dataset and achieved accuracy above 95%, especially with the Random Forest classifier.

We also designed the system in a modular way, following a microkernel-like approach — keeping the AI model in user space and only using a minimal scheduler extension in kernel space. This keeps the system safe, easy to maintain, and allows future improvements without making deep changes to the OS.

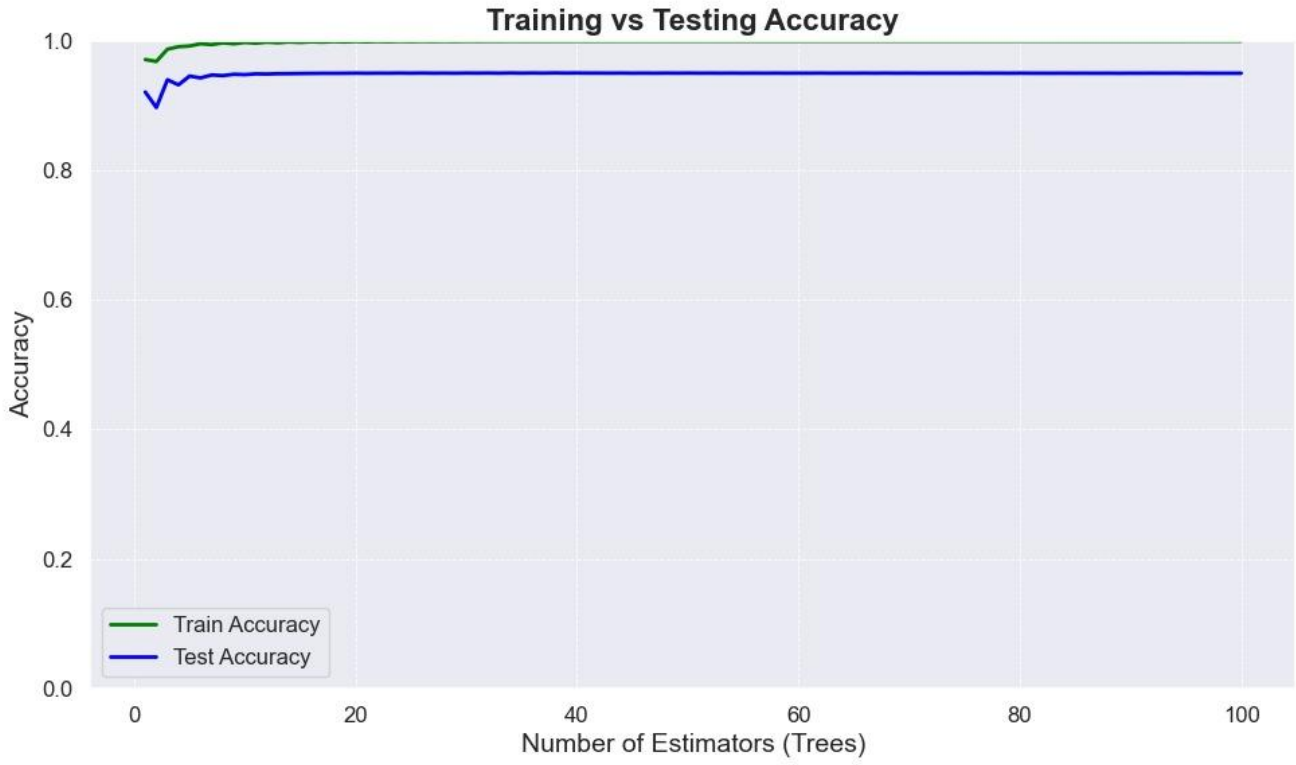
Below are the results of different models implemented:



Random Forest Confusion matrix (Highest Accuracy model)



Training vs Testing Accuracy report (generated using python)



Overall, we successfully implemented and tested the AI classification module, proved that it can predict task behavior effectively, and laid the groundwork for connecting it with the modified kernel scheduler.

CHAPTER 10: PLAN OF WORK FOR CAPSTONE PROJECT PHASE – 3

In Phase 2 of the project, we built and trained a machine learning model to classify tasks based on system-level data. For Phase 3, our focus will shift to improving this model so that it doesn't just classify tasks — but also predicts the optimal Smart Time Slice (STS) that each task should get. This value will be passed to the kernel scheduler to help it make better CPU allocation decisions.

The work in this phase will be organized as follows:

1. Predictive Model Enhancement
   We will extend our current classification model to also output a recommended STS value for each task. This will likely involve modifying the model to perform regression (instead of just classification), or building a separate regressor.

2. STS Prediction Evaluation
   We will train the STS predictor on real system data and evaluate how accurate and consistent it is. The goal is to match time slice values with task behavior in a way that improves performance and responsiveness.

3. Kernel Scheduler Integration
   Once the model is ready, we will modify or extend the Linux scheduler to accept the STS values. The AI component (in user space) will send predictions to the kernel through a secure channel — such as a Netlink socket or a system call wrapper.

4. Testing and Comparison
   We will test the full setup under different workload conditions. Our metrics will include average CPU usage, context switch overhead, task waiting time, and overall throughput. We will compare this against Linux's default scheduler (like CFS).

LIST OF FIGURES

REFERENCES/BIBLIOGRAPHY

[1] Hongjiu Lu, *ELF: From the Programmer's Perspective*, Technical Report, NYNEX Science and Technology, 500 Westchester Avenue, White Plains, NY 10604, USA, May 1995.
[2] Tigran Aivazian, *Linux Kernel 2.4 Internals*, The Linux Documentation Project, August 2002.
[3] Maurice Bach, *The Design of the Unix Operating System*, Pearson Education Asia, pp. 159-170, 2002.
[4] Fredrik Vraalsen, *Performance Contracts: Predicting and Monitoring Grid Application Behavior*, In Proceedings of the 2nd International Workshop on Grid Computing, November 2001.
[5] Richard Gibbons, *A Historical Application Profiler for Use by Parallel Schedulers*, Lecture Notes on Computer Science, Volume 1297, pp. 58-75, 1997.
[6] Hyok-Sung Choi and Hee-Chul Yun, *Context Switching and IPC Performance Comparison between uClinux and Linux on the ARM9-Based Processor*, Linux in Embedded Applications, January 2005. [Online].
[7] Warren Smith, Valerie Taylor, and Ian Foster, *Predicting Application Run Times Using Historical Information*, Job Scheduling Strategies for Parallel Processing, IPPS/SPDP'98 Workshop, March 1998.
[8] Kishore Kumar P. and Atul Negi, *Characterizing Process Execution Behaviour Using Machine Learning Techniques*, In DpROM Workshop Proceedings, HiPC 2004 International Conference, December 2004.
[9] S.R. Garner, *WEKA: The Waikato Environment for Knowledge Analysis*, In Proceedings of the New Zealand Computer Science Research Students Conference, pp. 57-64, 1995.
[10] Surkanya Suranauwarat and Hide Taniguchi, *The Design, Implementation, and Initial Evaluation of an Advanced Knowledge-Based Process Scheduler*, ACM SIGOPS Operating Systems Review, Volume 35, pp. 61-81, October 2001.
[11] Mark A. Hall, *Correlation-Based Feature Selection for Machine Learning*, Master's Thesis, Department of Computer Science, University of Waikato, pp. 7-9, 12-14, April 1999.
[12] Andrew Marks, *A Dynamically Adaptive CPU Scheduler*, Master's Thesis, Department of Computer Science, Santa Clara University, pp. 5-9, June 2003.
[13] Robert Love, *Linux Kernel Development*, 1st ed., Pearson Education, 2004.
[14] Daniel P. Bovet and Marco Cesati, *Understanding the Linux Kernel*, 2nd ed., O'Reilly & Associates, December 2002.
[15] Tom Mitchell, *Machine Learning*, 1st ed., McGraw-Hill International Edition, pp. 52-75, 154-183, 230-244, 1997.
[16] O. Duda and P.E. Hart, *Pattern Classification*, Wiley, New York, 1973.